D E E P T E C H N I C A L A U D I T

Auditoría técnica profunda

weltrade-webterminal-connector — conector MT5, motor de señales, PWA y laboratorio de ingeniería inversa

> Repositorio: github.com/albrth647-png/weltrade-webterminal-connector
>
> Commit auditado: 1e8bdfb (rama main, 21 commits)
>
> Fecha: 7 de noviembre de 2026
>
> Método: revisión manual línea a línea + Semgrep + njsscan + 14 pruebas dinámicas

Índice

1. Resumen ejecutivo.....1

2. Alcance, metodología y herramientas.....2

    2.1 Alcance.....2

    2.2 Metodología y herramientas.....2

3. Verificaciones ejecutadas.....3

4. Mapa de hallazgos.....4

5. Hallazgos críticos y altos.....4

    C1. Tormenta de reconexión exponencial en la PWA.....4

    A1. Sesiones WSS paralelas en motor-senales tras un login rechazado.....5

    A2. Contraseña de la cuenta demo publicada en la carpeta de evidencia.....5

    A3. Denegación de servicio por CPU en POST /importar sin autenticación.....6

6. Hallazgos medios.....6

    M1. La vela en formación se siembra como cerrada y nunca se corrige.....7

    M2. Las respuestas de velas (cmd 11) se entregan a todas las peticiones pendientes.....7

    M3. Escape de regex roto en la PWA: filtros incorrectos e interfaz colgada.....8

    M4. Endpoints de escritura sin autenticación.....8

    M5. Clave de sesión sin validación de longitud (reabierto).....9

    M6. Reintento infinito con credenciales inválidas (reabierto).....9

7. Hallazgos bajos.....9

8. Observaciones informativas.....11

9. Estado de la auditoría interna previa.....12

10. Recomendaciones priorizadas.....12

    P0 — Antes de volver a conectar (hoy mismo, menos de una hora).....12

    P1 — Antes de desplegar en Render o dejar la PWA instalada.....13

    P2 — Higiene y mantenimiento.....14

11. Límites de esta auditoría.....15

Nota: este índice se genera mediante códigos de campo. Tras editar el documento, haga clic derecho sobre el índice y elija "Actualizar campos" para refrescar la numeración de páginas.

1. Resumen ejecutivo

Este informe recoge la auditoría técnica profunda solicitada sobre el repositorio `weltrade-webterminal-connector` en su commit `1e8bdfb` (rama main, 21 commits). El proyecto comprende cuatro piezas: un conector Node.js al WebTerminal de MT5 de Weltrade obtenido por ingeniería inversa, un motor de señales con estrategia como documento externo, una PWA que porta el motor al navegador y un laboratorio de ingeniería inversa de bots .ex5 (scripts, Frida, Ghidra y flujos de CI).

El resultado global es **matizadamente positivo**. Los cimientos son sólidos: los 30 ficheros JavaScript pasan la verificación sintáctica, los 27 tests del repositorio pasan, Semgrep (reglas p/javascript y p/secrets) no emite ningún hallazgo y njsscan solo produce 8 avisos benignos ya documentados. La disciplina del proyecto es notable: sin secretos en el código, .env ignorado por git, validación estricta de documentos de estrategia y tests offline.

Sin embargo, la revisión manual línea a línea, combinada con pruebas dinámicas sobre el código real, descubrió **1 hallazgo crítico, 3 altos, 6 medios y 10 bajos** que la auditoría interna del 6 de noviembre de 2026 no cubrió, porque casi todos residen en código añadido después de esa auditoría o en caminos que no se simularon. Los tres más graves: la PWA multiplica exponencialmente sus reconexiones ante cualquier caída de red (se midieron 284.293 sockets en 1,2 segundos simulados, con la pestaña resultante inservible); la carpeta de evidencia publica la contraseña real de la cuenta demo en un binario que el escaneo anterior de credenciales no miró; y el endpoint público de importación admite una petición de 8 MB que consumiría horas de CPU por un coste cuadrático verificado.

Veredicto: **apto para uso personal local una vez aplicadas las correcciones P0** de la sección 10; **no apto** para despliegue público (Render) ni para dejarlo instalado unattended hasta resolver además las recomendaciones P1. Todas las correcciones son pequeñas (la mayoría, entre 3 y 15 líneas por fichero).

2. Alcance, metodología y herramientas

2.1 Alcance

Se leyó íntegramente el código de producción (conector, motor-senales, pwa), la documentación (README, LEEME, dossieres), los flujos de CI (.github/workflows), los scripts de laboratorio (scripts_re, incluidos el volcado de memoria bajo Wine, el gancho Frida y los scripts de Ghidra) y los artefactos de evidencia. El backtest de backtest_raptor/ se respetó congelado en modo solo lectura, tal y como exige el propio proyecto: no se re-ejecutó ni se modificó ningún fichero.

Un foco específico fue el código incorporado **después** de la auditoría interna documentada en docs/auditoria_proyecto.md (commits posteriores a su publicación): el importador de documentos, el gestor de estrategia, el servidor HTTP, la radiografía en el navegador (radio.js), el laboratorio MT5 sobre Wine y Windows, el kit Frida y los workflows de GitHub Actions.

2.2 Metodología y herramientas

Antes de auditar se investigó el ecosistema actual de asistentes de auditoría (skills y servidores MCP): la guía de Semgrep para ingenieros de seguridad sobre MCP y su servidor MCP oficial de análisis estático, njsscan/nodejsscan como SAST específico de Node.js, y las alternativas CodeQL y Snyk Code. De ese conjunto se instalaron y ejecutaron en el entorno de auditoría:

- `Semgrep` con los conjuntos `p/javascript` y `p/secrets` (0 hallazgos);
- `njsscan` (libsast + semgrep) sobre conector, motor y PWA (8 avisos WARNING benignos);
- `node --check` sobre los 30 ficheros JS (30/30 correctos);
- la suite de tests del propio repositorio (`test_estrategia.js` y `test_importador.js`, todos en verde);
- 14 pruebas dinámicas escritas ad hoc contra el código real (5 scripts Node conservados junto a este informe), incluyendo un WebSocket simulado para reproducir los bucles de reconexión;
- análisis forense en Python de los binarios de la carpeta evidencia/.

La revisión manual siguió el enfoque de la hoja de referencia de revisión de código de OWASP: mapa de entradas y flujos, criptografía, gestión de errores, estados de sesión, concurrencia y lógica de negocio. Dos hipótesis iniciales se **refutaron** al medirlas (sección 3) y por honestidad metodológica se documentan como descartadas: el ReDoS del patrón de sniffing no existe en V8 y el login no numérico no tumba el conector en Node 24.

3. Verificaciones ejecutadas

Todos los resultados de esta sección provienen de ejecuciones reales realizadas durante la auditoría; ninguna fila es una estimación. Las pruebas dinámicas P1 a P6 se ejecutaron contra los módulos reales del repositorio (importados con require) y el bucle de reconexión se replicó con el código exacto de app.js.

Tabla 1. Verificaciones y resultados

N.º	Verificación	Resultado
1	Sintaxis node --check de los 30 ficheros JS del repo	30/30 OK
2	Tests del repo: test_estrategia.js	Todos en verde (exit 0)
3	Tests del repo: test_importador.js	Todos en verde (exit 0)
4	Semgrep p/javascript + p/secrets (conector, motor, pwa)	0 hallazgos
5	njsscan (conector, motor-senales, pwa, scripts_re)	8 WARNING benignos (SHA-1 del protocolo, Math.random en fixtures de test)
6	P1: escape de regex de símbolos en la PWA (app.js)	CONFIRMADO: escape no operativo y SyntaxError con paréntesis o corchete suelto
7	P2: ReDoS del patrón de sniffing del importador	REFUTADO en V8: lineal (16-25 caracteres de relleno, menos de 1 ms)
8	P3: coste de buscarBloqueJson con JSON imposible	CONFIRMADO cuadrático: 8 KB a 51 ms, 31 KB a 676 ms, 125 KB a 10,5 s (factor 15,5 por cada 4 veces más entrada)
9	P4: bucle de reconexión de la PWA con red caída (WebSocket simulado)	CONFIRMADO: 284.293 sockets creados en 1,2 s (crecimiento exponencial)
10	P4d: login rechazado en motor-senales con su bucle de reconexión	CONFIRMADO: 70 sockets en 1 s frente a unos 23 esperables (multiplicación)
11	P5: vela en formación sembrada y luego cerrada	CONFIRMADO: la vela cerrada real se descarta y el buffer conserva el parcial
12	P6: dos peticiones de velas simultáneas (AAA y BBB)	CONFIRMADO: ambas promesas reciben la misma respuesta
13	Forense: evidencia/login_body.bin (911 B)	CONFIRMADO: contraseña demo de 8 caracteres ASCII extraíble en claro, cuenta 43396052 y servidor
14	writeUInt32LE con login no numérico (Node 24)	REFUTADO: no lanza excepción

4. Mapa de hallazgos

La tabla 2 resume los 20 hallazgos. Los identificadores C/A/M/B/I indican severidad crítica, alta, media, baja o informativa. La sección 9 compara además este mapa con la auditoría interna previa y declara qué hallazgos de aquella siguen abiertos.

Tabla 2. Resumen de hallazgos por severidad

ID	Severidad	Ubicación principal	Título
C1	Crítica	pwa/app.js + pwa/proto.js	Tormenta de reconexión exponencial en la PWA
A1	Alta	motor-senales/cliente_mt5.js + app.js	Sesiones WSS paralelas tras login rechazado
A2	Alta	evidencia/	Contraseña de la cuenta demo publicada en un binario
A3	Alta	motor-senales/importador.js + servidor.js	Denegación de servicio por CPU en POST /importar sin autenticación
M1	Media	estrategia.js, motor.js, app.js	La vela en formación se siembra como cerrada y nunca se corrige
M2	Media	cliente_mt5.js, proto.js	Las respuestas de velas (cmd 11) se entregan a todas las peticiones pendientes
M3	Media	pwa/app.js (línea 158)	Escape de regex roto: filtros incorrectos y interfaz colgada
M4	Media	motor-senales/servidor.js	Endpoints de escritura sin autenticación (escalada del hallazgo previo)
M5	Media	proto.js, conector.js, cliente_mt5.js	Clave de sesión sin validación de longitud (reabierto)
M6	Media	conector.js, app.js	Reintento infinito con credenciales inválidas (reabierto)
B1	Baja	pwa/app.js	Credenciales y token de Telegram en localStorage sin cifrar
B2	Baja	pwa/sw.js	Versión de caché del service worker incrementada a mano
B3	Baja	cliente_mt5.js vs proto.js	Divergencias de portaje nuevas entre las tres copias del protocolo
B4	Baja	motor-senales/app.js (línea 100)	Variable SIMBOLOS muerta: la lista de símbolos del documento no se aplica
B5	Baja	motor-senales/app.js	ticksUltimoMin solo se purga cuando alguien consulta /estado
B6	Baja	motor-senales/servidor.js	CSRF posible contra el panel local (POST simple sin comprobación de origen)
B7	Baja	motor-senales/ test_importador.js	Ruta absoluta del entorno del autor (/home/z/my-project/upload)

ID	Severidad	Ubicación principal	Título
B8	Baja	pwa/proto.js	Huella del login en UTF-16LE en la PWA frente a UTF-8 en Node
B9	Baja	conector, cliente_mt5, proto	El protocolo sigue triplicado en tres ficheros (causa raíz de B3 y B8)
B10	Baja	cliente_mt5.js, proto.js	Intervalo de ping huérfano si conectar() se invoca dos veces sin cerrar
I1	Info	informe	Semgrep 0 hallazgos; njsscan 8 avisos benignos
I2	Info	.github/workflows	Los inputs de workflow_dispatch quedan visibles en el resumen del run
I3	Info	protocolo	IV cero y clave estática son decisiones del servidor: la confidencialidad real es el TLS
I4	Info	repo	Sigue sin existir un test automatizado del parser de protocolo
I5	Info	weltrade-conector-termux.zip	El ZIP duplica conector/ y envejece fuera de control

5. Hallazgos críticos y altos

C1. Tormenta de reconexión exponencial en la PWA

Ubicación: `pwa/proto.js (_cerrar, sin guarda de idempotencia)` y `pwa/app.js (conectarInternal y el manejador de "cerrar").`

Tres piezas interactúan para producir crecimiento exponencial de conexiones. Primera: `_cerrar()` emite el evento "cerrar" cada vez que se invoca, sin comprobar si la sesión ya estaba apagada. Segunda: cuando la sesión cae por red, el evento llega dos veces para un mismo cierre real, porque `cerrar()` llama a `_cerrar()` de inmediato y después el navegador dispara `ws.onclose`, que vuelve a llamar a `_cerrar()`. Tercera: el manejador de "cerrar" de la PWA programa una reconexión, y `conectarInternal()` cierra al cliente anterior con `cliente.cerrar()`; como el cliente viejo conserva sus propios listeners, ese cierre emite otro "cerrar" y programa otra cadena adicional. Cada generación de reconexión duplica, pues, el número de cadenas pendientes.

Evidencia: replicando el bucle exacto de app.js con un WebSocket simulado que cae a los 60 ms, se crearon **284.293 sockets en 1,2 segundos**. Con el backoff real (3 a 30 s) la aritmética no mejora: las cadenas se duplican cada generación, de modo que en unos minutos hay

cientos de sesiones WSS simultáneas hacia el bróker y la pestaña deja de responder. Basta una única caída de red (o un login rechazado) con el motor encendido.

Corrección (pequeña): hacer idempotente `_cerrar()` en proto.js y cliente_mt5.js (guarda que retorne si la etapa ya es "apagado" y el evento ya se emitió), desconectar los listeners del cliente viejo en `conectarInternal()` (`removeAllListeners()` o una bandera `descartado`), y concentrar la reconexión en un único temporizador compartido en lugar de uno por evento.

A1. Sesiones WSS paralelas en motor-senales tras un login rechazado

Ubicación: `motor-senales/cliente_mt5.js (cerrar/_cerrar) y motor-senales/app.js (bucle de reconexión).`

El mismo defecto de doble emisión existe en el cliente Node: ante un login rechazado, `_onMensaje` llama a `this.cerrar()` (primera emisión) y el `onclose` posterior produce la segunda. El bucle de app.js reconecta la **misma instancia** en cada evento, de modo que un único rechazo deja dos WebSocket vivos contra el servidor con la misma cuenta; a partir de ahí cada caída del par multiplica las cadenas. Adicionalmente, `conectar()` sobrescribe `this._ping` sin limpiar el intervalo anterior, acumulando pings huérfanos (B10). Medición: 70 sockets creados en 1 segundo tras un rechazo, frente a los unos 23 que daría un bucle lineal sano.

Corrección: misma guarda de idempotencia que en C1, más no reconectar cuando el rechazo viene de credenciales (`dd ≠ 0` debe ser un estado terminal que exija intervención manual; véase M6), y limpiar el intervalo de ping al principio de `conectar()`.

A2. Contraseña de la cuenta demo publicada en la carpeta de evidencia

Ubicación: `evidencia/login_body.bin (911 B) y evidencia/session_decoded.json.`

El cuerpo de login de 912 bytes capturado durante la sesión real de ingeniería inversa quedó publicado en el repositorio. El campo de contraseña va en UTF-16LE desde el desplazamiento 4 y **no está cifrado** (el cifrado AES del frame se aplicó sobre el paquete completo, pero el fichero publicado es el cuerpo en claro, truncado en un byte). La auditoría interna declaró el escaneo de credenciales limpio porque solo examinó ficheros de texto (.js/.md/.json/.sh); el binario escapó al barrido. Un script de diez líneas recupera la contraseña íntegra de 8 caracteres ASCII, el número de cuenta (43396052) y el servidor (mt5.demo.weltrade.org). `session_decoded.json` añade la clave de sesión del handshake y fragmentos de cuerpos descifrados.

Impacto y corrección: aunque sea una cuenta demo, cualquiera con acceso al repositorio puede autenticarse con ella y consumir su sesión. Eliminar `login_body.bin` (y vaciar o anonimizar `session_decoded.json`), rotar la contraseña de esa cuenta demo inmediatamente, y si el repositorio es público valorar la purga del fichero del historial git. Añadir los binarios de evidencia a un escaneo de secretos (por ejemplo, decodificar ventanas UTF-16LE antes de buscar patrones).

A3. Denegación de servicio por CPU en POST /importar sin autenticación

Ubicación: `motor-senales/importador.js (buscarBloqueJson, línea 45) alcanzable desde motor-senales/servidor.js.`

Para documentos de texto sin bloque markdown, `buscarBloqueJson` intenta `JSON.parse` sobre todos los cortes posibles entre la primera llave abierta y cada llave cerrada hacia atrás. Con una entrada cuyo prefijo es JSON casi válido que solo falla al final, cada intento recorre casi toda la cadena: el coste total es cuadrático. Medición con el importador real: 8 KB a 51 ms, 31 KB a 676 ms y 125 KB a 10,5 s; el factor medido por cada cuatro veces más entrada es de 15,5, esencialmente el 16 teórico de $O(n^2)$. Extrapolado al máximo aceptado de 8 MB, una única petición consumiría del orden de **10 horas de CPU**. El endpoint es POST simple, sin token, sin límite de peticiones y desplegable en una URL pública de Render (que un pinger externo mantiene despierta, según recomienda el propio LEEME), de modo que una petición basta para degradar el servicio.

Corrección: limitar el número de intentos de parseo (por ejemplo, solo el corte entre la primera llave y la última, con un máximo de N intentos), impone un techo lineal; añadir un token compartido a las rutas de escritura (M4) y un límite de peticiones por IP. La PWA replica el mismo helper en radio.js, pero allí el coste lo paga el propio usuario sobre su fichero, y no es explotable por terceros.

6. Hallazgos medios

M1. La vela en formación se siembra como cerrada y nunca se corrige

Ubicación: `estrategia.js y motor.js (setHistoria), con la siembra de app.js y pwa/app.js.`

La respuesta del comando 11 incluye la vela actual en curso (el servidor la devuelve con su OHLC parcial en el instante de la petición). `setHistoria` la inserta en el buffer como si estuviera cerrada. Cuando esa vela cierra de verdad y llega por ticks, el guarda de duplicados de `onVelaCerrada` (`arr[última].t >= vela.t`) la rechaza porque ya existe una vela

con el mismo instante. La prueba dinámica P5 lo confirma: la vela real con máximos y mínimos verdaderos se descarta y el buffer conserva el parcial. Consecuencias: el ATR, las zonas y el sesgo quedan calculados sobre una vela truncada; y en la PWA el parcial se persiste en IndexedDB y el rango incremental de la próxima sesión empieza después de él, por lo que el dato corrupto sobrevive a las reconexiones. Una vela parcialmente formada tiene recorrido menor que la real, lo que tiende a subestimar el ATR y a deformar las bandas de toque exactamente en la vela más reciente, que es la que alimenta las señales.

Corrección: al sembrar, descartar la última vela si su periodo no está completo (`ult.t + tf_segundos*1000 > horaDeServidor`), o bien permitir que `onVelaCerrada` reemplace una vela del mismo instante que procede de siembra. Ambas variantes son de unas pocas líneas y eliminan el problema en las tres copias del motor.

M2. Las respuestas de velas (cmd 11) se entregan a todas las peticiones pendientes

Ubicación: `cliente_mt5.js y proto.js (_onMensaje, rama cmd 11; _pendientes11).`

La respuesta del servidor no identifica el símbolo pedido, y el código la difunde a **todas** las promesas pendientes del mapa. La prueba P6 lo demuestra: dos peticiones simultáneas (AAA y BBB) reciben ambas la misma vela. Hoy la siembra es estrictamente secuencial y el defecto no se manifiesta, pero `pedirVelas` es una API pública de la clase: cualquier solape futuro (resembrar tras activar una estrategia mientras corre la siembra inicial, o dos símbolos con el mismo nombre en mayúsculas distintas) mezclaría historiales de símbolos distintos sin ningún aviso. Además, el mapa está indexado por nombre de símbolo: una segunda petición del mismo símbolo reemplaza la pendiente y su promesa original queda sin resolver hasta el timeout.

Corrección: serializar las peticiones dentro del cliente (una cola que solo lance la siguiente al resolver la anterior) o rechazar una nueva petición mientras exista una pendiente; a medio plazo, correlacionar por el símbolo incrustado en la petición si el protocolo llega a devolverlo.

M3. Escape de regex roto en la PWA: filtros incorrectos e interfaz colgada

Ubicación: `pwa/app.js, línea 158 (crearCliente).`

La expresión `/[.*+?^${}()|[\\]\\]/g` cierra la clase de caracteres antes de tiempo y el `replace` resulta ser un no operativo: la prueba P1 muestra que "Step.5", "Crash+1" o "SFX*Vol" pasan sin escapar. Dos efectos verificados: símbolos con metacaracteres compilan regex que casan con nombres equivocados (el filtro se queda corto o largo

silenciosamente), y un símbolo con paréntesis o corchete desbalanceado (por ejemplo "FX(2" o "Boom [500") lanza SyntaxError dentro de `crearCliente()` durante el click de Conectar: la interfaz queda clavada en "conectando..." sin recuperación salvo recargar. La copia de Node (motor-senales/app.js, línea 42) tiene la expresión correcta: es otra divergencia de portaje.

Corrección: copiar la expresión de motor-senales (`/[.*+?^${}()|[\]\\]/g`) y, como refuerzo, envolver `crearCliente()` en try/catch que reporte el error al log visible.

M4. Endpoints de escritura sin autenticación

Ubicación: `motor-senales/servidor.js (/importar, /estrategia/activar, /estrategia/restaurar).`

La auditoría interna señaló la lectura de /estado sin autenticación (su M6). Desde entonces el servidor ganó rutas de **escritura**: cualquiera que alcance la URL (local o Render) puede reemplazar la estrategia activa, restaurar la integrada o subir documentos al directorio de importados, y en combinación con A3 también tumbar el proceso. Los nombres de fichero se sanea correctamente (no hay traversía), pero la superficie de escritura pública es nueva y más grave que la de lectura.

Corrección: token opcional por variable de entorno (por ejemplo `ESTADO_TOKEN`) exigido en todas las rutas POST y en /estado; sin token configurado, enlazar el servidor a 127.0.0.1 salvo que haya proxy delante.

M5. Clave de sesión sin validación de longitud (reabierto)

Ubicación: `pwa/proto.js (134), conector/conector.js (80), motor-senales/cliente_mt5.js (106).`

Es el hallazgo M3 de la auditoría interna y **sigue sin corregirse en ninguno de los tres clientes**: `body.slice(66, 98)` acepta cualquier longitud menor y el fallo posterior de cifrado queda tragado por el try vacío, con el síntoma de conexión viva que no entrega datos. Sigue siendo la corrección más barata con mejor ratio de la lista: una guarda de 3 líneas con log y cierre por cliente.

M6. Reintento infinito con credenciales inválidas (reabierto)

Ubicación: `conector/conector.js (bucle de 3 s) y pwa/app.js (backoff).`

El M4 interno no se aplicó y ahora interactúa con C1 y A1: un rechazo de credenciales no se distingue de una caída de red, reintenta indefinidamente y, con los defectos de doble emisión, además multiplica las sesiones. Corrección combinada: contador de rechazos consecutivos (máximo 3-5), estado terminal visible ("credenciales rechazadas") y parada hasta acción manual; el backoff queda reservado a fallos de transporte.

7. Hallazgos bajos

Los diez hallazgos bajos son deudas de operación o higiene; ninguno corrompe señales por sí solo, pero varios alimentan los anteriores.

- **B1 — Credenciales en localStorage.** Cuenta, contraseña y token de Telegram viven sin cifrar en la PWA (M1 interno, abierto). Mitigación barata: sessionStorage por defecto y casilla de recordar.
- **B2 — Caché del service worker a mano.** Mejoró de v1 a v3 (alguien la incrementó), pero el mecanismo sigue siendo manual: un cambio de proto.js sin bump deja a los instalados con código viejo indefinidamente (M2 interno, parcialmente atendido).
- **B3 — Nuevas divergencias de portaje.** Detectadas tres: _recalcAtr de Node conserva el ATR anterior con historial insuficiente mientras la PWA lo pone a cero; _cerrar de Node no limpia el temporizador debounce de las peticiones de velas (la PWA sí); y la radiografía del importador publica campos de cabecera (0x58, 0x6C) que radio.js omite. La paridad 91/91 verificada en la auditoría interna no cubre estos caminos.
- **B4 — Variable muerta y contrato roto.** motor-senales/app.js calcula SIMBOLOS (línea 100) y nunca la usa; el filtro de suscripción sale solo de MS_SIMBOLOS del entorno. El comentario de .env.ejemplo ("si el documento trae simbolos, ese manda") es falso: la lista del documento se ignora incluso tras reiniciar.
- **B5 — Purga diferida de ticks.** En motor-senales, ticksUltimoMin solo se depura cuando alguien consulta /estado; si nadie consulta, crece sin límite (del orden de un millón de entradas al día con quince símbolos). La PWA sí purga cada 2 segundos.
- **B6 — CSRF contra el panel local.** Los POST del servidor son peticiones simples sin comprobación de origen: una página maliciosa abierta en el mismo navegador puede activar estrategias contra http://localhost:3000 mientras el motor corre. La corrección de M4 (token) la cubre de paso.
- **B7 — Ruta de entorno del autor en un test.** test_importador.js referencia /home/z/my-project/upload/HobbieCode_GoldRush.ex5; fuera de esa máquina el test 7 se salta en silencio y la cobertura baja sin avisar. Conviene copiar la muestra al repo (si la licencia lo permite) o usar una ruta relativa con aviso.
- **B8 — Huella del login con codificación distinta.** La PWA hashea termux;login;fecha en UTF-16LE y Node en UTF-8 (B1 interno, abierto). El servidor no la valida, así que el impacto hoy es nulo; se mantiene por fidelidad del portaje.
- **B9 — Protocolo triplicado.** Cripto, login y parseo viven por duplicado en conector.js, cliente_mt5.js y proto.js (B2 interno, abierto); es la causa raíz que sigue produciendo B3 y B8 y que hizo posible C1 en una copia y no en las otras.

- **B10 — Ping huérfano.** conectar() asigna un setInterval de ping sin limpiar el anterior; combinado con A1 (reconexiones sobre la misma instancia) acumula intervalos que disparan pings repetidos cada 8 segundos.

8. Observaciones informativas

- **I1 — SAST limpio.** Semgrep (p/javascript + p/secrets) emite cero hallazgos en el código de producción; njsscan emite ocho WARNING: tres usos de SHA-1 que corresponden a la huella del protocolo del servidor (documentado como no validada por él), dos Math.random en fixtures de test y dos console.error en emisores de eventos; nada accionable.
- **I2 — Inputs de CI visibles.** En los workflows del laboratorio MT5, los valores pasados por workflow_dispatch (login y contraseña demo) aparecen en los metadatos del run, visibles para quien pueda ver el repositorio; los secrets MT5_LOGIN/MT5_PASSWORD sí quedan enmascarados. Recomendación operativa: usar siempre la vía de secrets.
- **I3 — Criptografía del protocolo.** IV de ceros, AES-CBC y clave estática ofuscada son decisiones del servidor de MetaQuotes/Weltrade, no elecciones locales; la confidencialidad real descansa en el TLS. No es un defecto del proyecto, pero tampoco debe presentarse como cifrado fuerte (I1 interno).
- **I4 — Sin test del parser de protocolo.** El M5 interno sigue abierto: login de 912 B, catálogo de 526 B, ticks de 50 B y velas de 48 B carecen de prueba automatizada; las pruebas dinámicas de esta auditoría (P5, P6) demuestran que ese camino produce defectos reales y merece cobertura permanente.
- **I5 — ZIP duplicado.** weltrade-conector-termux.zip sigue versionado a mano en la raíz (B9 interno, abierto): desincronizado del código vivo, alguien lo instalará creyendo que es lo último.

9. Estado de la auditoría interna previa

La auditoría interna del 6 de noviembre de 2026 (docs/auditoria_proyecto.md) fue seria y sus verificaciones aritméticas del backtest son correctas; este trabajo no las reabre. Su principal límite no fue de calidad sino de recorte temporal: el 40 por ciento de los commits actuales (11 de 21) y varias rutas críticas llegaron después, y las dinámicas de reconexión no se simularon. La tabla 3 resume el destino de sus hallazgos.

Tabla 3. Hallazgos de la auditoría interna y su estado actual

ID interno	Título abreviado	Estado en este audit
M1	Credenciales en localStorage	Abierto (B1)
M2	Caché del SW versionada a mano	Parcial: subió a v3, mecanismo manual (B2)
M3	Clave de sesión sin validación de 32 B	Abierto, sin corregir (M5)
M4	Reintento infinito con credenciales inválidas	Abierto y agravado por C1/A1 (M6)
M5	Sin test del parser de protocolo	Abierto (I4); esta auditoría aporta las pruebas como base
M6	/estado sin autenticación	Ampliado: ahora también escritura sin auth (M4)
B1	Huella UTF-16 frente a UTF-8	Abierto (B8)
B2	Protocolo triplicado	Abierto (B9), con 3 divergencias nuevas (B3)
B3	R fijo nominal frente a realizado	Sin cambios: backtest congelado, se respeta
B7	Reconexiones cuenta cierres manuales	Abierto (cosmético)
B8	Metadatos iOS	Parcial: apple-touch-icon añadido; falta la meta de app-capable
B9	ZIP duplicado	Abierto (I5)
B10	Instalador sin checksum	Abierto
Laguna	No se simuló la reconexión	Origen de C1 y A1: la doble emisión de "cerrar" no era visible sin simular
Laguna	Escaneo de secretos solo textual	Origen de A2: login_body.bin no pasó por el barrido
Laguna	Coste del importador no medido	Origen de A3: buscarBloqueJson llegó sin techo de intentos

10. Recomendaciones priorizadas

P0 — Antes de volver a conectar (hoy mismo, menos de una hora)

- C1: guarda de idempotencia en _cerrar de proto.js y cliente_mt5.js; desconectar listeners del cliente viejo en conectarInternal; temporizador de reconexión único.

- A1: la misma guarda en Node y no reconectar ante dd distinto de 0; limpiar el intervalo de ping al inicio de conectar().
- A2: eliminar evidencia/login_body.bin, anonimizar session_decoded.json y rotar la contraseña de la cuenta demo 43396052.
- M5: validar que la clave de sesión mide 32 bytes en los tres clientes, con log y cierre.

P1 — Antes de desplegar en Render o dejar la PWA instalada

- A3: techo de intentos en buscarBloqueJson (por ejemplo, 32 cortes máximos) y token en las rutas POST.
- M4: ESTADO_TOKEN en /importar, /estrategia/activar, /estrategia/restaurar y /estado; sin token, enlazar a 127.0.0.1.
- M3: portar la expresión de escape correcta de motor-senales a pwa/app.js y proteger crearCliente con try/catch.
- M1: descartar la vela no cerrada al sembrar (o reemplazar por mismo instante) en estrategia.js, motor.js y la siembra de ambas apps.
- M2: serializar pedirVelas dentro del cliente (cola de una petición).
- M6: contador de rechazos de login con estado terminal.

P2 — Higiene y mantenimiento

- B4: eliminar la variable muerta o aplicar de verdad doc.simbolos al filtro del cliente; corregir el comentario de .env.ejemplo.
- B5: purgar ticksUltimoMin con el mismo intervalo de 2 s que la PWA.
- B1: sessionStorage por defecto con casilla de recordar; B2: bump automático de caché con hash de contenido.
- B9: extraer el protocolo a un módulo único del que deriven Node y navegador; con ello mueren B3 y B8.
- B7: sustituir la ruta absoluta del test por una muestra versionada o relativa; I2: usar solo secrets en los workflows.
- I4: convertir las pruebas dinámicas P5 y P6 en un test_protocolo.mjs permanente de CI.

Responsable sugerido: el propio maintenedor del repositorio, con un PR por severidad (un PR P0 con cuatro commits pequeños es realista para una sesión). Verificación de cada corrección: las pruebas dinámicas conservadas junto a este informe (verificar_hallazgos.js y sus rondas) sirven como regresión; en particular, la prueba de la tormenta debe quedar en 1 socket por caída tras la corrección de C1.

11. Límites de esta auditoría

- No se estableció conexión real contra los servidores de Weltrade: las pruebas dinámicas usan el código real de los clientes con un WebSocket simulado; el comportamiento del servidor real (encabezados Origin, candados por IP) queda fuera.
- No se auditó el servidor de Weltrade ni la legalidad o los términos de servicio del acceso por protocolo revertido: corresponde al propietario del proyecto.
- El backtest permanece congelado en modo documental, según el mandato del propio repositorio: sus cifras no se re-ejecutaron ni se discuten más allá de lo ya documentado.
- La refutación del ReDoS (P2) vale para V8 (Node 24 y Chrome actuales); otros motores de regex podrían comportarse peor, pero todos los runtimes del proyecto son V8 o navegadores Blink.
- Los scripts de laboratorio (scripts_re) se revisaron como herramientas de uso manual en runners efímeros; un uso distinto (por ejemplo, credenciales reales en el lab) exigiría otra revisión.